

ggvmap: Voronoi map treemaps for ggplot2 in pure R

Loukas Theodosiou

loukesio@gmail.com

Abstract Treemaps communicate part-of-whole relationships by mapping quantities to areas. Voronoi treemaps generalise the familiar rectangular layout to arbitrary convex boundaries and organic, polygon-shaped cells, but the existing R implementations delegate the layout either to a JavaScript/D3 runtime or to compiled CGAL code, which complicates installation, reproducibility, and integration with the ggplot2 ecosystem. Here I present ggvmmap, an R package that computes Voronoi map treemaps in pure R – its only hard dependency is ggplot2 – and renders them as ordinary ggplot2 objects. ggvmmap supports hierarchical (grouped) layouts, a grammar of pipe-able annotation layers (rings, flags, images, value labels), automatic small-cell label handling, and interactive hover maps. The tessellations it produces are exact power diagrams whose defining geometric properties are enforced by property-based tests. Benchmarks against `voronoiTreemap` (D3) and `WeightedTreemaps` (CGAL) show that ggvmmap has the simplest installation and the shortest code for the same plot, is the fastest of the measurable implementations below about fifty cells, and trades runtime – governed by an explicit, user-visible iteration budget – at larger sizes.

Highlights

- Voronoi map treemaps as native ggplot2 layers, in pure R with a single hard dependency (ggplot2).
- Hierarchical (grouped) layouts plus a grammar of pipe-able annotation verbs: rings (band or labelled arc), country flags, images, and value labels.
- Automatic small-cell handling: per-cell label autoscaling and area-based label suppression.
- Verified correctness: the layout is an exact power diagram, enforced by property-based tests on every build.
- Convergence is never silent: printing the layout reports the iteration count, the residual area error, and a converged flag, so an unfaithful map is caught before it is plotted.

1 Introduction

Part-of-whole structure is among the most common shapes of scientific data: gene families within a genome, reads within a sequencing library, budget positions within an economy. Treemaps display such data by making the area of a cell proportional to its weight. The classic rectangular treemap is easy to compute but produces high-aspect-ratio slivers and a strongly grid-like appearance; *Voronoi treemaps* [1] instead partition an arbitrary convex polygon into organic cells, improving both aesthetics and area perception, and are the layout behind many widely shared data-journalism infographics.

Computing a Voronoi treemap requires iteratively adapting an additively weighted Voronoi diagram – a power diagram [2] – until every cell’s area matches its target share [3]. In R, two implementations exist: **voronoiTreemap**, which wraps the D3/JavaScript layout in an `htmlwidget` [4], and **WeightedTreemaps**, which computes the layout in compiled C++ against the CGAL geometry library [5]. Both work well, but neither produces a ggplot2 object, and both carry installation burdens – a browser runtime in one case, CGAL compilation in the other – that complicate scripted, reproducible pipelines. There is no pure-R, ggplot2-native option. **ggvmap** fills that gap, following the design precedent of `tidyplots` [6] in offering a small, consistent, pipe-able interface on top of ggplot2 [7].

2 Results

2.1 A grammar of Voronoi maps

Figure 1 summarises the interface. A single computation function, `voronoi_map()`, takes weights (optionally with labels and a grouping vector) and returns the tessellation; `ggvmap()` renders it as a ggplot; and a family of

`vm_add_*()` verbs – `vm_add_ring()`, `vm_add_labels()`, `vm_add_flags()`, `vm_add_images()` – add annotation layers through the native pipe, each reading the layout from the plot object and returning a new plot, so the layers chain without intermediate variables (Figure 1 A, C). `ggvmap()` also accepts raw weights directly, collapsing computation and rendering into one call.

Appearance is controlled at three levels (Figure 1 D): the map (palette, fill mapping, clip shape, font family), the group (per-group border colours for emphasis), and the cell (per-cell font faces, label colours, and label selection). Arguments accept either a single value or a vector named by cell or group, so emphasising one region of an otherwise uniform map is a one-argument change. Because every result is a `ggplot`, titles, themes, and extensions from the wider `ggplot2` ecosystem (e.g. markdown titles via `ggtext`, custom fonts via `showtext`) apply unchanged.

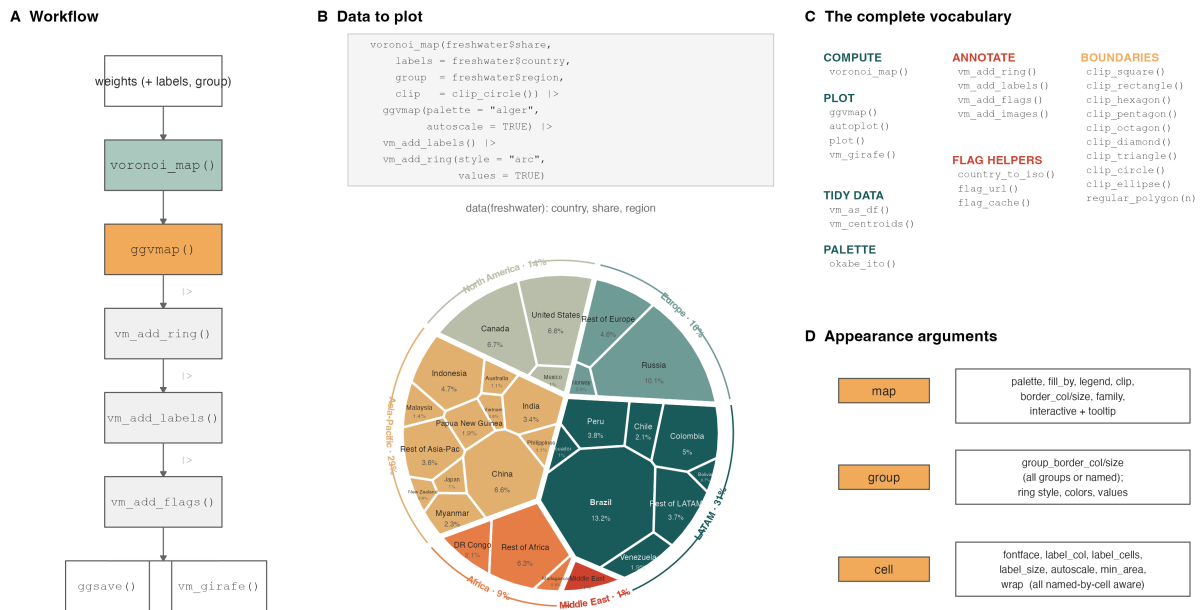


Figure 1: Design of `ggvmap`. (A) The workflow: a weight vector (with optional labels and a grouping variable) flows through `voronoi_map()`, which computes the tessellation, into `ggvmap()`, which renders it; annotation verbs are chained with R's native pipe, and the result is written to file with `ggsave()` or made interactive with `vm_girafe()`. (B) From data to plot: the nine lines of code shown produce the finished infographic below them from the bundled freshwater dataset (columns country, share, region; each country's share of global renewable freshwater resources, 2022 [8]). Every cell is a country with area proportional to its share, colour and sector encode the region, values sit beneath the names (shrunk or hidden in the smallest cells), and the outer arc ring carries each region's total share. (C) The complete exported vocabulary, grouped by role: one computation function; four output functions; four pipe-able annotation verbs; accessors that return tidy data frames for custom `ggplot2` work; the colour and flag helpers; and the ten convex boundary constructors. (D) Appearance arguments by the level they act on – whole map, group, or single cell. Every cell-level argument also accepts a vector named by cell label, so emphasising one cell (bold face, different colour, its own label) is a one-argument change with no data manipulation.

2.2 The layout is a diagnosable object

Between computation and rendering sits a deliberate checkpoint. The recommended workflow is two steps – compute the layout, inspect it, and only then plot:

```

vm <- voronoi_map(freshwater$share, labels = freshwater$country,
  group = freshwater$region, clip = clip_circle(),
  seed = 5, max_iter = 80)

vm
#> Voronoi Map
#> hierarchical | 6 groups | 30 cells
#> 30 cells | 17 iterations | convergence: 1.509%
#> Converged: TRUE

```

Printing the object reports the three facts that cannot be read off the picture. First, the Converged flag: the layout is an iterative optimisation with a finite budget (`max_iter`), so it can stop before cell areas match their targets – and an unconverged map still renders and looks entirely plausible, which is precisely why the check must precede the plot. If the flag is `FALSE`, the defining promise of a treemap (area proportional to value) is broken, and the remedy is a larger `max_iter` or a different seed. Second, the residual area error, which quantifies how faithful the converged areas are – a figure a manuscript can report directly. Third, the iteration count, which shows how much of the budget was consumed and therefore how much margin the layout had.

The separation also reflects where the cost and the randomness live. The layout is the expensive, seed-dependent step; rendering is cheap and deterministic. Computing the tessellation once, verifying it, and passing the same object to every downstream call – `ggvmap()`, the annotation verbs, the tidy accessors, `vm_girafe()` – guarantees that all outputs describe one verified layout instead of silently recomputing (and potentially re-randomising) it per plot.

2.3 One dataset, many maps

Figure 2 shows the same dataset across the package’s layout and annotation space: nine convex boundary shapes, a hierarchical layout with one sector per region and per-group border emphasis, the two ring styles (filled band and labelled arc with group shares), continuous fill by weight, and flag annotations. Labels in small cells are automatically shrunk (`autoscale`) or suppressed (`min_area`), which keeps maps with long-tailed weight distributions legible without manual tuning.

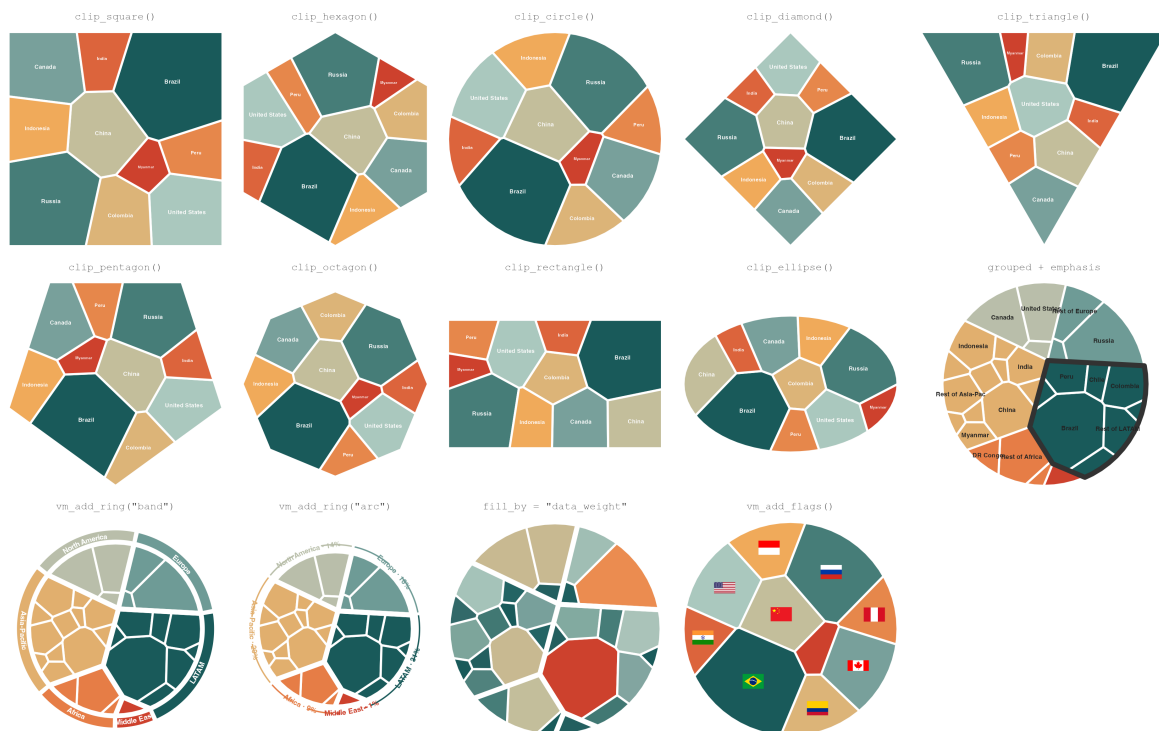


Figure 2: Gallery: one dataset, fourteen maps. Rows 1–2: the ten largest freshwater-holding countries laid out on each of the nine built-in convex boundaries – the identical weight vector fills any convex shape with area-true cells, so the boundary is a free design choice. Row 3, left to right: the full 30-row dataset in a hierarchical layout (one sector per region) with the LATAM region singled out by a dark border (`group_border_col = c(LATAM = "#333333")`); the filled "band" ring style, where each segment spans the angular extent of its region’s sector; the thin "arc" ring style with each region’s share appended to its label (`values = TRUE`); continuous fill mapped to cell weight (`fill_by = "data_weight"`), turning the map into a treemap-heatmap hybrid; and national flags resolved automatically from the country names (`vm_add_flags()`). Throughout, labels in small cells shrink (`autoscale = TRUE`) or disappear below an area threshold (`min_area`), keeping long-tailed data legible without manual tuning.

2.4 Use cases

Figure 3 sketches four disciplinary applications, each fully scripted in the repository: gene families by genome share, genome-wide association hits per chromosome, RNA-seq library composition (with the contaminant fraction outlined for emphasis), and household expenditure shares. The data in this figure are simulated but realistic; the scripts are drop-in templates for real data.

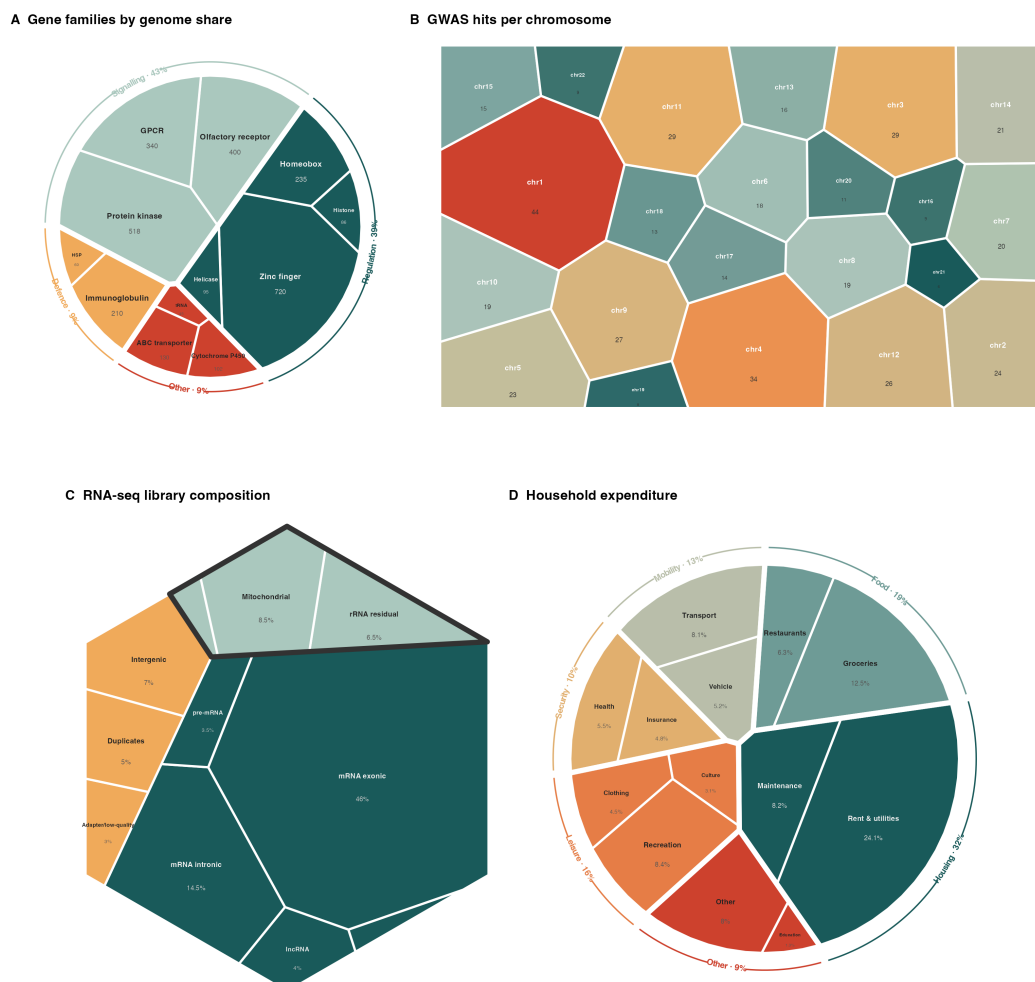


Figure 3: Use cases across disciplines. All data are simulated but realistic, and each panel's complete script ships in `paper/figures/fig3_usecases.R` as a drop-in template. (A) Genomics: twelve large gene families sized by gene count and grouped by functional class; the arc ring reports each class's share of the depicted families, and light-on-dark label colours keep the dark Regulation sector readable. (B) Genetics: simulated genome-wide association hits per human chromosome (weights roughly proportional to chromosome length) on a rectangular boundary, with continuous fill restating each cell's weight so hot chromosomes stand out twice – by area and by colour. (C) Transcriptomics: where the reads of an RNA-seq library end up, grouped into signal, contaminant, and noise classes; the contaminant class is outlined dark for emphasis and fractions below 2% of the library lose their labels (`min_area`). (D) Social science: household expenditure shares grouped by need category, the same hierarchy-plus-arc-ring pattern as panel A applied to survey-style data.

2.5 Benchmarks

Figure 4 compares `ggvmap` with `voronoiTreemap` and `WeightedTreemaps` on four axes, each package at its default settings. *Install complexity* (Figure 4 A): `ggvmap` needs 17 recursively resolved CRAN packages and no compiled code, versus 36 packages plus CGAL compilation for `WeightedTreemaps`; `voronoiTreemap` resolves 44 packages and runs its layout in a browser. *Code length* (Figure 4 B): the same grouped treemap takes the fewest words,

characters, and function calls in ggvmmap. *Runtime* (Figure 4 C) is size-dependent. On small maps ggvmmap is the faster of the two measurable implementations – 68 ms versus 664 ms at $n = 10$ and 2.5 s versus 4.5 s at $n = 50$ – because it avoids the fixed setup cost of the compiled pipeline. Beyond that, pure R pays its price: 33.4 s versus 5.5 s at $n = 100$, and a 30-fold gap at $n = 500$ (14.5 min vs 29.5 s). Treemaps typically summarise tens of categories, where ggvmmap is competitive or faster; for maps of many hundreds of cells, WeightedTreemaps is the right tool.

Accuracy (Figure 4 D) turns out to measure convergence, not intrinsic layout quality. At $n = 10$ and $n = 50$ the layout converges below the default 1% area-error threshold within 32 and 63 iterations, giving 2% and 2% mean relative cell-area error. The $n = 100$ and $n = 500$ instances exhaust the default budget of 200 iterations before converging; the stopped layouts show 57% and 12% mean error, and the $n = 100$ draw – whose lognormal weights span a 128-fold range – is the hardest of the four. This is an iteration-budget artefact, not a ceiling: rerunning the same $n = 100$ instance with `max_iter = 500` converges after 383 iterations to 2.3% mean error (63 s). ggvmmap is deliberately transparent about this: the returned object carries a converged flag and the iteration count. WeightedTreemaps shows the same budget-dependence under its own defaults (5% to 42% across sizes), and both implementations concentrate their largest relative errors in the smallest cells, where the target area is near zero and a large *relative* error is visually invisible. `voronoiTreemap` computes its layout in the browser, so R-side runtime and accuracy are not measurable; it appears only in panels A–B.

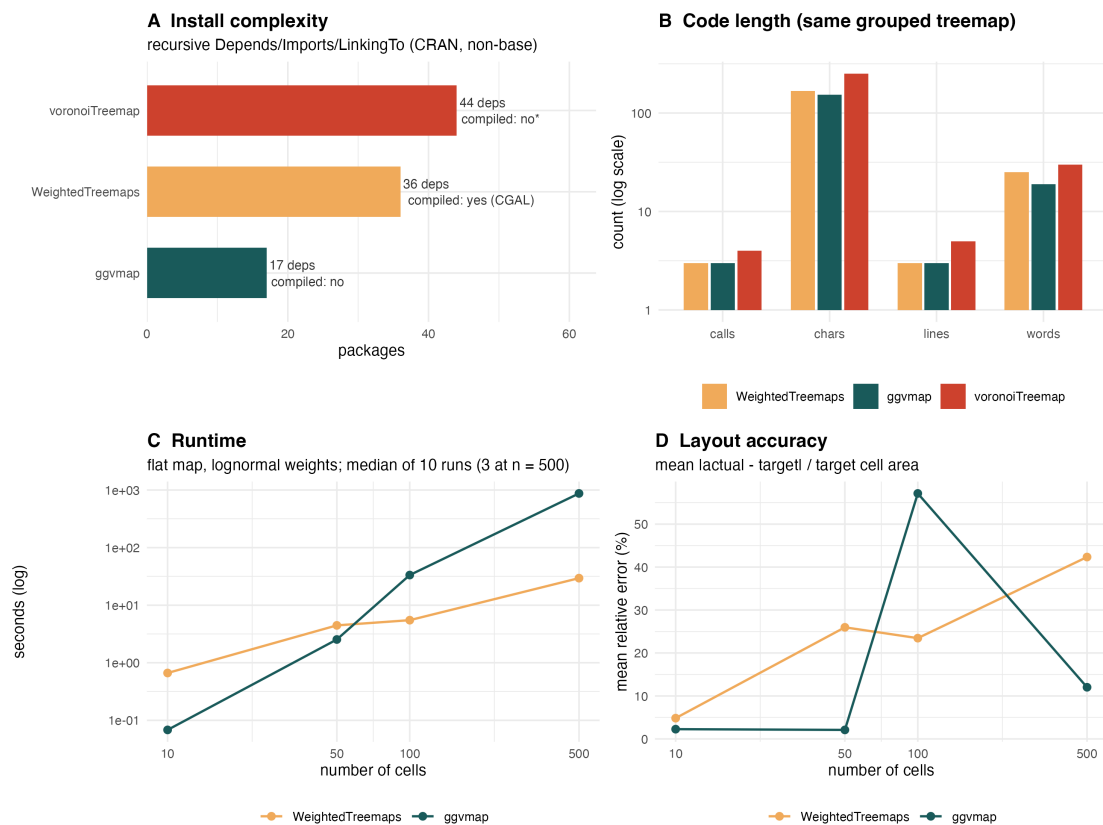


Figure 4: Benchmarks against the two existing R implementations, all packages at default settings; the full protocol is in Methods and paper/figures/fig4_benchmark.R. (A) Install complexity: recursively resolved CRAN dependencies (Depends, Imports, LinkingTo; base packages excluded). ggvmmap’s 17 are ggplot2 and its tree, with no compiled code; WeightedTreemaps adds CGAL compilation; voronoiTreemap runs on the htmlwidgets/D3 stack. (B) Code length for the same grouped treemap of data(freshwater): lines, words, characters, and function calls of the minimal script in each package (log scale; lower is better). (C) Median layout runtime on flat maps of lognormal(3, 1) weights, log-log axes: ggvmmap (teal) is fastest up to $n = 50$, then the compiled implementation (orange) pulls ahead – 33 s vs 5.5 s at $n = 100$ and 14.5 min vs 29 s at $n = 500$. (D) Mean relative cell-area error, $|actual - target| / target$. The ggvmmap points at $n = 100$ and $n = 500$ are layouts stopped by the default 200-iteration budget (the returned converged flag is FALSE); the same $n = 100$ instance converges to 2.3% mean error given `max_iter = 500`. Panels C–D omit voronoiTreemap, whose layout runs in the browser and is not measurable from R.

3 Conclusion

ggvmap makes Voronoi map treemaps a first-class citizen of the ggplot2 ecosystem: a one-line install, a small pipeable grammar, verified geometry, and publication-style annotation layers. The pure-R implementation trades raw speed at large cell counts for zero installation friction and full composability – a trade that favours the typical treemap, which summarises tens of categories, not thousands. Its main current limitations are the quadratic per-iteration cost of the clipping loop, which makes maps beyond a few hundred cells slow, and the absence of leader lines for labelling cells too small to hold text. Both are planned: spatial indexing restricts clipping to a cell’s actual neighbours and is expected to reduce the large-map runtime by an order of magnitude without leaving pure R, and callout labels are deferred until they can be added without new hard dependencies.

4 Methods

4.1 Interface design

The interface follows three rules, adopting the design vocabulary of tidyplots [6] and ggplot2 [7]: (i) one verb per action – `voronoi_map()` computes, `ggvmap()` renders, `vm_add_*()` annotate; (ii) every function both accepts and returns the natural object of its stage (weights to layout to plot), so calls compose with the pipe; (iii) arguments degrade gracefully from scalars to named vectors, so emphasis is expressed by naming cells or groups rather than by restructuring data. Annotation layers re-attach the layout object to the returned plot, which is what makes the chain stateless for the user.

4.2 Algorithm

The layout follows [3]. Each cell is a site with a position and an additive power weight; the plane is partitioned by the power diagram [2], computed here in pure R by sequential half-plane clipping of the convex boundary. Each iteration moves every site to its cell’s centroid (position adaptation) and multiplicatively adjusts its power weight by the ratio of target to actual area (weight adaptation), until the maximum relative area error falls below a convergence threshold. Intuitively, the algorithm is a room full of balloons: each balloon drifts toward the centre of its territory and inflates or deflates until it holds exactly its share of the room. Hierarchical maps solve the group-level map first, then run the same algorithm inside each group polygon – a valid nested power diagram. Weights are initialised proportional to target areas and sites by farthest-point sampling, which removes degenerate starts. Iteration stops when the maximum relative area error falls below `convergence_ratio` (default 1%) or after `max_iter` iterations (default 200); the returned object records both the iteration count and a converged flag, so an unconverged layout is never silent – rerunning with a larger `max_iter` or a different seed resolves it.

4.3 Correctness

The test suite enforces the defining properties of the output rather than snapshots: every point of a fine grid lies in the cell of its minimum-power-distance site; cell areas sum to the clip area to machine precision with no gaps or overlaps; every cell is convex; and each hierarchical subcell lies inside its group polygon (`tests/testthat/test-correctness.R`). These property-based tests run on every build.

4.4 Dependencies

ggvmap Imports only ggplot2 (plus the base packages `grDevices`, `stats`, `utils`). Optional features degrade gracefully behind `requireNamespace()` guards: `ggimage` (flags, images), `ggiraph` (interactivity), `geomtextpath` (curved ring labels), `magick` and `png` (image caching).

4.5 Benchmarking

Benchmarks (Figure 4) were run with `bench` [9]: flat maps of $n = 10, 50, 100, 500$ `lognormal(3, 1)` weights, median of 10 iterations (3 at $n = 500$), on a single laptop core, each package at its default settings (for `ggvmap`: `convergence_ratio = 0.01`, `max_iter = 200`, `min_weight_ratio = 0.01`). Layouts that reach `max_iter` unconverged are reported as measured; the text notes where a larger budget converges them. Dependency counts are recursive `Depends`, `Imports` and `LinkingTo` on CRAN, excluding base packages. Code-length metrics count lines, words, characters, and function calls of the minimal code producing the same grouped treemap in each package. Layout accuracy is the mean over cells of $|actual - target| / target$ area, taken from each package’s reported per-cell areas. The full script is `paper/figures/fig4_benchmark.R`; results are archived in `paper/figures/benchmark_results.csv`.

4.6 Data availability

ggvmap is available at <https://github.com/loukesio/ggvmap> under the MIT licence. The freshwater dataset ships with the package (FAO Aquastat via World Bank, 2022 [8]). All figures in this paper are generated by the scripts in `paper/figures/` and rendered by `paper/render.R`.

Acknowledgements

This package grew out of the author’s Stack Overflow question “Make a circular Voronoi diagram in R” and owes much to the inspiration of Allan Cameron’s answer there.

References

Bibliography

- [1] M. Balzer and O. Deussen, “Voronoi Treemaps”, in *Proceedings of the IEEE Symposium on Information Visualization (InfoVis)*, 2005, pp. 49–56. doi: 10.1109/INFVIS.2005.1532128.
- [2] F. Aurenhammer, “Power Diagrams: Properties, Algorithms and Applications”, *SIAM Journal on Computing*, vol. 16, no. 1, pp. 78–96, 1987, doi: 10.1137/0216006.
- [3] A. Nocaj and U. Brandes, “Computing Voronoi Treemaps: Faster, Simpler, and Resolution-independent”, *Computer Graphics Forum*, vol. 31, no. 3, pp. 855–864, 2012, doi: 10.1111/j.1467-8659.2012.03078.x.
- [4] M. Li, E. Mills, and UOB HDRUK, “voronoiTreemap: Voronoi Treemaps with Added Interactivity by Shiny”. [Online]. Available: <https://cran.r-project.org/package=voronoiTreemap>
- [5] M. Jahn and Y. Leifels, “WeightedTreemaps: Generate and Plot Voronoi or Sunburst Treemaps from Hierarchical Data”. [Online]. Available: <https://cran.r-project.org/package=WeightedTreemaps>
- [6] J. B. Engler, “Tidyplots Empowers Life Scientists With Easy Code-Based Data Visualization”, *iMeta*, vol. 4, p. e70018, 2025, doi: 10.1002/imt2.70018.
- [7] H. Wickham, *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York, 2016. [Online]. Available: <https://ggplot2.tidyverse.org/>
- [8] Food and Agriculture Organization of the United Nations, “AQUASTAT: FAO’s Global Information System on Water and Agriculture -- Renewable Internal Freshwater Resources”. [Online]. Available: <https://data.worldbank.org/indicator/ER.H2O.INTR.K3>
- [9] J. Hester and D. Vaughan, “bench: High Precision Timing of R Expressions”. [Online]. Available: <https://cran.r-project.org/package=bench>